

AI Agent 工程教程

AI Agent 工程学习系统 · 本地可维护版本

源文件：[AI_AGENT_ENGINEERING_TUTORIAL.md](#)

目录

AI Agent 工程教程	1
前言：这不是一门只讲模型的课程	7
使用方式	7
贯穿全书的五个判断	7
全书路线	7
第一篇：基础工程	7
本篇要解决的问题	7
能力地图	7
学习方法	8
本篇核心观点	8
1. 从边界开始	8
2. 从确定性部分开始	8
3. 从失败路径证明成熟度	8
本篇项目连接	8
本篇章节	8
第 1 章：Python 与本地开发环境	8
第 2 章：Git、HTTP 与 FastAPI	9
第 3 章：SQL 与 PostgreSQL	9
第 4 章：工程化 API 阶段项目	10
本篇练习顺序	11
本篇完成标准	11
延伸阅读	11
第二篇：LLM 应用	11
本篇要解决的问题	11
能力地图	11
学习方法	12
本篇核心观点	12
1. 从边界开始	12
2. 从确定性部分开始	12
3. 从失败路径证明成熟度	12
本篇项目连接	12
本篇章节	12
第 5 章：LLM 基础与 Prompt 结构	12

第 6 章：Structured Output 与 Tool Calling	13
第 7 章：UX Research Copilot	13
第 8 章：可靠性、安全与阶段交付	14
本篇练习顺序	14
本篇完成标准	15
延伸阅读	15
第三篇：RAG 知识库	15
本篇要解决的问题	15
能力地图	15
学习方法	15
本篇核心观点	16
1. 从边界开始	16
2. 从确定性部分开始	16
3. 从失败路径证明成熟度	16
本篇项目连接	16
本篇章节	16
第 9 章：文档解析与 Chunking	16
第 10 章：Embedding 与向量检索	17
第 11 章：Hybrid Search、Reranking 与引用	17
第 12 章：RAG 评测与数据隔离	18
本篇练习顺序	18
本篇完成标准	19
延伸阅读	19
第四篇：Agent 工作流	19
本篇要解决的问题	19
能力地图	19
学习方法	19
本篇核心观点	20
1. 从边界开始	20
2. 从确定性部分开始	20
3. 从失败路径证明成熟度	20
本篇项目连接	20
本篇章节	20
第 13 章：Agent、Workflow 与 ReAct	20
第 14 章：LangGraph 状态工作流	21

第 15 章：记忆、Human-in-the-loop 与 MCP	21
第 16 章：智能设计评审 Agent 交付	22
本篇练习顺序	22
本篇完成标准	22
延伸阅读	23
第五篇：生产化	23
本篇要解决的问题	23
能力地图	23
学习方法	23
本篇核心观点	23
1. 从边界开始	23
2. 从确定性部分开始	23
3. 从失败路径证明成熟度	24
本篇项目连接	24
本篇章节	24
第 17 章：测试与 Agent Evaluation	24
第 18 章：可观测性、成本与性能	24
第 19 章：权限、安全与数据治理	25
第 20 章：Docker、CI/CD 与部署	25
本篇练习顺序	26
本篇完成标准	26
延伸阅读	26
第六篇：作品集与求职	27
本篇要解决的问题	27
能力地图	27
学习方法	27
本篇核心观点	27
1. 从边界开始	27
2. 从确定性部分开始	27
3. 从失败路径证明成熟度	27
本篇项目连接	28
本篇章节	28
第 21 章：作品集工程化整理	28
第 22 章：Python、API 与数据库面试	28
第 23 章：RAG、Agent 与系统设计面试	29

第 24 章：简历、Demo 与投递 29

本篇练习顺序 30

本篇完成标准 30

延伸阅读 30

项目实战篇 31

P1：设计需求分析助手 31

 这个项目训练什么 31

 用户和业务流程 31

 核心工程能力 31

 必须证明的结果 31

 风险边界 31

 实施入口 32

 项目完成标准 32

P2：设计规范知识库 Agent 32

 这个项目训练什么 32

 用户和业务流程 32

 核心工程能力 32

 必须证明的结果 33

 风险边界 33

 实施入口 33

 项目完成标准 33

P3：智能设计评审 Agent 34

 这个项目训练什么 34

 用户和业务流程 34

 核心工程能力 34

 必须证明的结果 34

 风险边界 34

 实施入口 35

 项目完成标准 35

P4：供应商风险审查 Agent 35

 这个项目训练什么 35

 用户和业务流程 35

 核心工程能力 35

 必须证明的结果 36

风险边界 36

实施入口 36

项目完成标准 36

结语：从学习者到交付者 36

前言：这不是一门只讲模型的课程

这套教程面向有设计背景和少量代码基础、希望进入 AI Agent 工程岗位的学习者。学习目标不是记住框架 API，而是完成一条真实的产品交付链：理解需求、设计数据和接口、接入模型、检索知识、编排工具、处理失败、评测效果、保障安全、部署服务，并把这些工作讲清楚。

使用方式

- 1. 先看 教程目录 和 总纲。
- 2. 按章节阅读本书，每章结束后打开对应周计划。
- 3. 每天用 weeks/week-XX/days/day-XX.md 执行，代码写入对应练习目录。
- 4. 每 4 周完成一个阶段项目，不把“读完”当作“完成”。
- 5. 通过 update_plan.py 记录完成任务、时间、问题和个人笔记。

贯穿全书五个判断

- 这是一个真实用户问题，还是为了展示技术而造的 Demo？
- 哪些规则应该由代码控制，哪些内容适合交给模型？
- 如果模型、工具、数据库或用户输入失败，系统如何保持可控？
- 我用什么数据和指标证明系统变好了？
- 招聘方能否按 README 运行并理解我的取舍？

全书路线

基础工程 → LLM 应用 → RAG 知识库 → Agent 工作流 → 生产化 → 求职交付

第一篇：基础工程

本篇要解决的问题

先把想法变成可运行、可测试、可持久化的服务。

能力地图

周	主题	交付物	详细入口
W01	Python 与本地开发环境	可运行的需求数据清洗命令行工具	周计划
W02	Git、HTTP 与 FastAPI	带三个接口的需求助手 API 原型	周计划
W03	SQL 与 PostgreSQL	可持久化需求记录的 API	周计划
W04	工程化 API 阶段项目	设计需求结构化助手 API v1	周计划

学习方法

本篇不要按“看完概念再集中编码”的方式学习。每个主题都要走完：概念理解 → 最小例子 → 接入项目 → 错误处理 → 测试或评测 → README 表达。

本篇核心观点

1. 从边界开始

先写清用户、输入、输出、权限和不做事项，再决定模型、框架或数据库。技术选择服务于业务流程，不是为了堆名词。

2. 从确定性部分开始

能用代码、SQL、Schema 或状态机明确表达的规则，不交给模型自由发挥。模型负责语言理解和候选生成，系统负责校验、权限、持久化和终止。

3. 从失败路径证明成熟度

每个阶段至少演示一个失败：空输入、错误类型、服务超时、检索无结果、工具拒绝、人工驳回或部署失败。真实工程能力体现在失败后系统仍然可控。

本篇项目连接

本篇主要推进：projects/project-01-tool-calling/。打开项目的 README.md、TASKS.md 和 ACCEPTANCE_CRITERIA.md，将章节知识转成可验收交付物。

本篇章节

第 1 章：Python 与本地开发环境

这一章解决什么：Python 3.12、虚拟环境、类型提示、函数、类、模块与调试。

为什么岗位需要：能独立阅读和修改 Agent 项目的 Python 代码，并建立可复现环境。

必须掌握

- 虚拟环境
- 类型提示
- 函数与模块
- 异常信息阅读

需要理解

- 类与数据模型
- 包结构

章节实践

完成“可运行的需求数据清洗命令行工具”。先运行 Day 01 最小示例，再按 本周计划 接入项目，最后使用 验收清单 检查。

常见风险

容易把时间耗在语法细节；以可运行的小程序为主。

面试自检

1. 为什么需要虚拟环境？
2. 类型提示能解决什么问题？

第 2 章：Git、HTTP 与 FastAPI

这一章解决什么：Git 工作流、HTTP、JSON、REST API、FastAPI 路由与测试。

为什么岗位需要：真实 Agent 通常通过 API 接收任务、返回结构化结果，并用 Git 协作。

必须掌握

- 提交与分支
- HTTP 方法和状态码
- JSON
- FastAPI 路由

需要理解

- REST 约束
- 依赖注入

章节实践

完成“带三个接口的需求助手 API 原型”。先运行 Day 01 最小示例，再按 本周计划 接入项目，最后使用 验收清单 检查。

常见风险

只会复制接口代码但不理解输入输出契约。

面试自检

1. PUT 与 PATCH 有何区别？
2. 如何设计可维护的 API？

第 3 章：SQL 与 PostgreSQL

这一章解决什么：关系模型、SQL、PostgreSQL、CRUD、事务与迁移。

为什么岗位需要：Agent 需要可靠保存用户、任务、工具调用和评测结果。

必须掌握

- 表与主键
- SELECT/INSERT/UPDATE
- CRUD

- 数据库连接配置

需要理解

- 索引
- 事务
- 迁移

章节实践

完成“可持久化需求记录的 API”。先运行 Day 01 最小示例，再按 本周计划 接入项目，最后使用 验收清单 检查。

常见风险

本地数据库配置可能占用过多时间；先完成最小闭环。

面试自检

1. 索引何时会失效？
2. 事务解决什么问题？

第 4 章：工程化 API 阶段项目

这一章解决什么：分层、日志、异常处理、测试、环境变量与 README。

为什么岗位需要：把零散知识收束为可展示、可运行、可解释的小项目。

必须掌握

- 配置隔离
- 日志
- 错误响应
- API 测试

需要理解

- 服务层
- 测试替身

章节实践

完成“设计需求结构化助手 API v1”。先运行 Day 01 最小示例，再按 本周计划 接入项目，最后使用 验收清单 检查。

常见风险

功能堆积导致无法验收；严格按接受标准收尾。

面试自检

1. 如何设计统一错误格式？
2. 怎样证明接口可维护？

本篇练习顺序

- 1. 先完成 W01 的 Day 01 最小示例。
- 2. 每周完成 WEEK_PLAN.md 和 CHECKLIST.md。
- 3. 每天把代码放在对应 exercises/day-XX/，把个人理解写在每日文件的保护区域。
- 4. 本篇结束时完成 W04 的项目里程碑，并写 REVIEW.md。

本篇完成标准

- [] 能不看教程解释本篇的核心概念。
- [] 能运行对应周的最小代码模板。
- [] 能说明一次失败、排查过程和改进。
- [] 项目 README、测试、验收和学习日志已更新。
- [] 能回答本篇周计划中的面试问题。

延伸阅读

- 24 周路线图
- 完整目录
- 本篇对应项目

第二篇：LLM 应用

本篇要解决的问题

再让模型按照结构化契约完成可靠任务，并能调用工具。

能力地图

周	主题	交付物	详细入口
W05	LLM 基础与 Prompt 结构	可复用的访谈摘要 Prompt 套件	周计划
W06	Structured Output 与 Tool Calling	能稳定输出研究洞察 JSON 的工具调用原型	周计划
W07	UX Research Copilot	可处理一份访谈记录的 Copilot	周计划
W08	可靠性、安全与阶段交付	UX Research Copilot v1	周计划

学习方法

本篇不要按“看完概念再集中编码”的方式学习。每个主题都要走完：概念理解 → 最小例子 → 接入项目 → 错误处理 → 测试或评测 → README 表达。

本篇核心观点

1. 从边界开始

先写清用户、输入、输出、权限和不做事项，再决定模型、框架或数据库。技术选择服务于业务流程，不是为了堆名词。

2. 从确定性部分开始

能用代码、SQL、Schema 或状态机明确表达的规则，不交给模型自由发挥。模型负责语言理解和候选生成，系统负责校验、权限、持久化和终止。

3. 从失败路径证明成熟度

每个阶段至少演示一个失败：空输入、错误类型、服务超时、检索无结果、工具拒绝、人工驳回或部署失败。真实工程能力体现在失败后系统仍然可控。

本篇项目连接

本篇主要推进：projects/project-01-tool-calling/。打开项目的 README.md、TASKS.md 和 ACCEPTANCE_CRITERIA.md，将章节知识转成可验收交付物。

本篇章节

第 5 章：LLM 基础与 Prompt 结构

这一章解决什么：Token、上下文窗口、消息角色、指令层级与 Prompt 模板。

为什么岗位需要：理解模型边界，才能设计稳定的 Agent 输入与输出。

必须掌握

- Token
- 上下文
- 系统指令
- Prompt 模板

需要理解

- 采样参数
- 上下文压缩

章节实践

完成“可复用的访谈摘要 Prompt 套件”。先运行 Day 01 最小示例，再按本周计划接入项目，最后使用验收清单检查。

常见风险

把 Prompt 调整当成玄学；必须记录输入、版本和结果。

面试自检

1. 上下文窗口与记忆有什么区别？
2. 如何减少 Prompt 歧义？

第 6 章：Structured Output 与 Tool Calling

这一章解决什么：JSON Schema、结构化输出、函数调用、参数验证。

为什么岗位需要：Agent 与业务系统协作依赖机器可验证的结构化契约。

必须掌握

- JSON Schema
- Pydantic 模型
- 工具定义
- 参数校验

需要理解

- 模式兼容
- 失败修复

章节实践

完成“能稳定输出研究洞察 JSON 的工具调用原型”。先运行 Day 01 最小示例，再按 本周计划 接入项目，最后使用 验收清单 检查。

常见风险

只验证成功路径，忽略模型返回缺字段或错误类型。

面试自检

1. Structured Output 与普通 JSON Prompt 有何差别？
2. 工具参数为何需要校验？

第 7 章：UX Research Copilot

这一章解决什么：研究资料解析、洞察提取、工具编排与人工确认。

为什么岗位需要：将设计研究能力转化为 AI 产品与 Agent 工程优势。

必须掌握

- 任务拆分
- 工具调用
- 证据引用
- 人工确认

需要理解

- 批处理
- 提示词版本

章节实践

完成“可处理一份访谈记录的 Copilot”。先运行 Day 01 最小示例，再按 本周计划 接入项目，最后使用 验收清单 检查。

常见风险

输出看似合理但缺乏原文证据。

面试自检

1. 何时必须加入人工确认？
2. 如何避免模型编造洞察？

第 8 章：可靠性、安全与阶段交付

这一章解决什么：超时、重试、降级、成本记录、Prompt Injection 与 Demo。

为什么岗位需要：生产应用必须在模型超时、拒答和恶意输入下仍可控。

必须掌握

- 超时
- 指数退避
- 降级
- 输入隔离
- 成本日志

需要理解

- 幂等性
- 内容过滤

章节实践

完成“UX Research Copilot v1”。先运行 Day 01 最小示例，再按 本周计划 接入项目，最后使用 验收清单 检查。

常见风险

只做正常演示，没有故障和安全场景。

面试自检

1. 重试何时会放大故障？
2. Prompt Injection 如何进入系统？

本篇练习顺序

1. 先完成 W05 的 Day 01 最小示例。

- 2. 每周完成 WEEK_PLAN.md 和 CHECKLIST.md。
- 3. 每天把代码放在对应 exercises/day-XX/，把个人理解写在每日文件的保护区域。
- 4. 本篇结束时完成 W08 的项目里程碑，并写 REVIEW.md。

本篇完成标准

- [] 能不看教程解释本篇的核心概念。
- [] 能运行对应周的最小代码模板。
- [] 能说明一次失败、排查过程和改进。
- [] 项目 README、测试、验收和学习日志已更新。
- [] 能回答本篇周计划中的面试问题。

延伸阅读

- 24 周路线图
- 完整目录
- 本篇对应项目

第三篇：RAG 知识库

本篇要解决的问题

让答案基于可检索、可引用、可评测的外部知识。

能力地图

周	主题	交付物	详细入口
W09	文档解析与 Chunking	设计规范文档摄取流水线	周计划
W10	Embedding 与向量检索	支持元数据过滤的检索 API	周计划
W11	Hybrid Search、Reranking 与引用	可返回引用片段的问答链路	周计划
W12	RAG 评测与数据隔离	设计规范知识库 Agent v1	周计划

学习方法

本篇不要按“看完概念再集中编码”的方式学习。每个主题都要走完：概念理解 → 最小例子 → 接入项目 → 错误处理 → 测试或评测 → README 表达。

本篇核心观点

1. 从边界开始

先写清用户、输入、输出、权限和不做事项，再决定模型、框架或数据库。技术选择服务于业务流程，不是为了堆名词。

2. 从确定性部分开始

能用代码、SQL、Schema 或状态机明确表达的规则，不交给模型自由发挥。模型负责语言理解和候选生成，系统负责校验、权限、持久化和终止。

3. 从失败路径证明成熟度

每个阶段至少演示一个失败：空输入、错误类型、服务超时、检索无结果、工具拒绝、人工驳回或部署失败。真实工程能力体现在失败后系统仍然可控。

本篇项目连接

本篇主要推进：`projects/project-02-rag-knowledge-base/`。打开项目的 `README.md`、`TASKS.md` 和 `ACCEPTANCE_CRITERIA.md`，将章节知识转成可验收交付物。

本篇章节

第 9 章：文档解析与 Chunking

这一章解决什么：文档加载、清洗、切块策略、重叠与元数据。

为什么岗位需要：RAG 质量首先取决于进入索引的数据结构。

必须掌握

- 解析
- Chunk
- 重叠
- Metadata

需要理解

- 语义切块
- 表格处理

章节实践

完成“设计规范文档摄取流水线”。先运行 Day 01 最小示例，再按 本周计划 接入项目，最后使用 验收清单 检查。

常见风险

直接套固定字符数，破坏章节语义。

面试自检

1. Chunk 过大或过小有什么影响？

2. 元数据如何帮助检索？

第 10 章：Embedding 与向量检索

这一章解决什么：Embedding、相似度、向量库、过滤与召回。

为什么岗位需要：把自然语言问题映射到可搜索的知识空间。

必须掌握

- Embedding
- Top-k
- 相似度
- 过滤

需要理解

- 向量索引
- 召回率

章节实践

完成“支持元数据过滤的检索 API”。先运行 Day 01 最小示例，再按 本周计划 接入项目，最后使用 验收清单 检查。

常见风险

只看单条结果，不建立代表性查询集。

面试自检

1. Embedding 模型更换会带来什么影响？
2. Top-k 如何选择？

第 11 章：Hybrid Search、Reranking 与引用

这一章解决什么：关键词+向量混合、重排、查询改写、证据引用。

为什么岗位需要：提高长尾查询命中率，并让答案可核查。

必须掌握

- Hybrid Search
- Reranking
- Query Rewrite
- Citation

需要理解

- 融合分数
- 无答案策略

章节实践

完成“可返回引用片段的问答链路”。先运行 Day 01 最小示例，再按 本周计划 接入项目，最后使用 验收清单 检查。

常见风险

把检索分数当作答案正确率。

面试自检

1. 为什么需要 Reranker？
2. 引用怎样降低业务风险？

第 12 章：RAG 评测与数据隔离

这一章解决什么：Golden Dataset、检索评测、答案评测、租户隔离与阶段交付。

为什么岗位需要：用可重复的评测而非主观感觉迭代 RAG。

必须掌握

- 测试集
- Recall@k
- 引用正确性
- 权限过滤

需要理解

- 评测偏差
- 回归测试

章节实践

完成“设计规范知识库 Agent v1”。先运行 Day 01 最小示例，再按 本周计划 接入项目，最后使用 验收清单 检查。

常见风险

用模型自评代替全部人工核验。

面试自检

1. RAG 应分别评测哪些环节？
2. 权限过滤应放在哪一层？

本篇练习顺序

1. 先完成 W09 的 Day 01 最小示例。
2. 每周完成 WEEK_PLAN.md 和 CHECKLIST.md。
3. 每天把代码放在对应 exercises/day-XX/，把个人理解写在每日文件的保护区域。
4. 本篇结束时完成 W12 的项目里程碑，并写 REVIEW.md。

本篇完成标准

- [] 能不看教程解释本篇的核心概念。
- [] 能运行对应周的最小代码模板。
- [] 能说明一次失败、排查过程和改进。
- [] 项目 README、测试、验收和学习日志已更新。
- [] 能回答本篇周计划中的面试问题。

延伸阅读

- 24 周路线图
- 完整目录
- 本篇对应项目

第四篇：Agent 工作流

本篇要解决的问题

把单次调用升级为有状态、有边界、可人工接管的多步骤流程。

能力地图

周	主题	交付物	详细入口
W13	Agent、Workflow 与 ReAct	可解释的多步骤任务图	周计划
W14	LangGraph 状态工作流	设计评审工作流骨架	周计划
W15	记忆、Human-in-the-loop 与 MCP	带审批节点和外部工具的工作流	周计划
W16	智能设计评审 Agent 交付	智能设计评审 Agent v1	周计划

学习方法

本篇不要按“看完概念再集中编码”的方式学习。每个主题都要走完：概念理解 → 最小例子 → 接入项目 → 错误处理 → 测试或评测 → README 表达。

本篇核心观点

1. 从边界开始

先写清用户、输入、输出、权限和不做事项，再决定模型、框架或数据库。技术选择服务于业务流程，不是为了堆名词。

2. 从确定性部分开始

能用代码、SQL、Schema 或状态机明确表达的规则，不交给模型自由发挥。模型负责语言理解和候选生成，系统负责校验、权限、持久化和终止。

3. 从失败路径证明成熟度

每个阶段至少演示一个失败：空输入、错误类型、服务超时、检索无结果、工具拒绝、人工驳回或部署失败。真实工程能力体现在失败后系统仍然可控。

本篇项目连接

本篇主要推进：`projects/project-03-agent-workflow/`。打开项目的 `README.md`、`TASKS.md` 和 `ACCEPTANCE_CRITERIA.md`，将章节知识转成可验收交付物。

本篇章节

第 13 章：Agent、Workflow 与 ReAct

这一章解决什么：Agent 边界、确定性工作流、ReAct、状态与终止条件。

为什么岗位需要：选择正确的自动化形式，避免把所有逻辑都交给模型。

必须掌握

- Workflow
- ReAct
- State
- 终止条件

需要理解

- 计划与执行
- 状态持久化

章节实践

完成“可解释的多步骤任务图”。先运行 Day 01 最小示例，再按 本周计划 接入项目，最后使用 验收清单 检查。

常见风险

循环没有预算或终止条件。

面试自检

1. 何时不应该使用 Agent？

2. ReAct 的主要风险是什么？

第 14 章：LangGraph 状态工作流

这一章解决什么：节点、边、条件分支、循环、检查点与错误路由。

为什么岗位需要：用显式状态图构建可调试、可恢复的 Agent 流程。

必须掌握

- 节点
- 条件边
- 状态更新
- Checkpoint

需要理解

- 子图
- 并行分支

章节实践

完成“设计评审工作流骨架”。先运行 Day 01 最小示例，再按 本周计划 接入项目，最后使用 验收清单 检查。

常见风险

状态字段无所有权约定，节点互相覆盖。

面试自检

1. 状态图相比链式调用有什么优势？
2. 如何防止无限循环？

第 15 章：记忆、Human-in-the-loop 与 MCP

这一章解决什么：短期/长期记忆、人工审批、MCP 工具接入。

为什么岗位需要：在自动化效率与人的控制权之间建立清晰边界。

必须掌握

- 短期记忆
- 长期记忆
- 审批中断
- MCP

需要理解

- 记忆淘汰
- 工具权限

章节实践

完成“带审批节点和外部工具的工作流”。先运行 Day 01 最小示例，再按 本周计划 接入项目，最后使用 验收清单 检查。

常见风险

把全部聊天历史当作长期记忆。

面试自检

1. 记忆和数据库记录有什么不同？
2. MCP 解决了什么集成问题？

第 16 章：智能设计评审 Agent 交付

这一章解决什么：多步骤评审、证据、重试、日志、Tracing 与成本。

为什么岗位需要：形成体现设计背景、工程能力和产品判断的核心作品。

必须掌握

- 完整状态流
- 人工确认
- 错误恢复
- 追踪

需要理解

- 成本预算
- 离线回放

章节实践

完成“智能设计评审 Agent v1”。先运行 Day 01 最小示例，再按 本周计划 接入项目，最后使用 验收清单 检查。

常见风险

为了炫技使用过多 Agent，增加不确定性。

面试自检

1. 如何解释你的 Agent 架构取舍？
2. 怎样证明评审结果可靠？

本篇练习顺序

1. 先完成 W13 的 Day 01 最小示例。
2. 每周完成 WEEK_PLAN.md 和 CHECKLIST.md。
3. 每天把代码放在对应 exercises/day-XX/，把个人理解写在每日文件的保护区域。
4. 本篇结束时完成 W16 的项目里程碑，并写 REVIEW.md。

本篇完成标准

- [] 能不看教程解释本篇的核心概念。
- [] 能运行对应周的最小代码模板。

- [] 能说明一次失败、排查过程和改进。
- [] 项目 README、测试、验收和学习日志已更新。
- [] 能回答本篇周计划中的面试问题。

延伸阅读

- 24 周路线图
- 完整目录
- 本篇对应项目

第五篇：生产化

本篇要解决的问题

用测试、评测、安全、观测和部署把 Demo 变成产品。

能力地图

周	主题	交付物	详细入口
W17	测试与 Agent Evaluation	项目统一测试与评测框架	周计划
W18	可观测性、成本与性能	可观测的 Agent 服务	周计划
W19	权限、安全与数据治理	项目安全基线与威胁模型	周计划
W20	Docker、CI/CD 与部署	可部署的 Agent Web 产品	周计划

学习方法

本篇不要按“看完概念再集中编码”的方式学习。每个主题都要走完：概念理解 → 最小例子 → 接入项目 → 错误处理 → 测试或评测 → README 表达。

本篇核心观点

1. 从边界开始

先写清用户、输入、输出、权限和不做事项，再决定模型、框架或数据库。技术选择服务于业务流程，不是为了堆名词。

2. 从确定性部分开始

能用代码、SQL、Schema 或状态机明确表达的规则，不交给模型自由发挥。模型负责语言理解和候选生成，系统负责校验、权限、持久化和终止。

3. 从失败路径证明成熟度

每个阶段至少演示一个失败：空输入、错误类型、服务超时、检索无结果、工具拒绝、人工驳回或部署失败。真实工程能力体现在失败后系统仍然可控。

本篇项目连接

本篇主要推进：projects/project-04-capstone/。打开项目的 README.md、TASKS.md 和 ACCEPTANCE_CRITERIA.md，将章节知识转成可验收交付物。

本篇章节

第 17 章：测试与 Agent Evaluation

这一章解决什么：单元测试、集成测试、Golden Dataset、回归评测。

为什么岗位需要：团队需要知道每次修改是否让系统变好或变坏。

必须掌握

- 单元测试
- 集成测试
- Golden Dataset
- 回归

需要理解

- 评测抽样
- 评分 Rubric

章节实践

完成“项目统一测试与评测框架”。先运行 Day 01 最小示例，再按 本周计划 接入项目，最后使用 验收清单 检查。

常见风险

只测模型输出文本完全相等。

面试自检

1. 非确定性系统怎样做回归测试？
2. Golden Dataset 如何维护？

第 18 章：可观测性、成本与性能

这一章解决什么：结构化日志、Tracing、指标、缓存、限流与成本分析。

为什么岗位需要：定位慢、贵、错发生在哪个模型、工具和状态节点。

必须掌握

- Trace ID
- 延迟指标

- 缓存

- 限流

需要理解

- 采样

- 成本分摊

章节实践

完成“可观测的 Agent 服务”。先运行 Day 01 最小示例，再按 本周计划 接入项目，最后使用 验收清单 检查。

常见风险

日志包含隐私或完整 Prompt。

面试自检

1. 缓存 LLM 响应有哪些风险？
2. Agent 应记录哪些关键指标？

第 19 章：权限、安全与数据治理

这一章解决什么：认证授权、租户隔离、密钥、Prompt Injection、防泄漏。

为什么岗位需要：AI 应用会连接高权限工具，安全边界必须先于自动化能力。

必须掌握

- 最小权限
- Secret 管理
- 数据隔离
- 输入输出过滤

需要理解

- 审计日志
- 威胁建模

章节实践

完成“项目安全基线与威胁模型”。先运行 Day 01 最小示例，再按 本周计划 接入项目，最后使用 验收清单 检查。

常见风险

只在 Prompt 中写禁止事项，不做代码级权限。

面试自检

1. Tool Calling 的主要攻击面是什么？
2. 如何做租户级数据隔离？

第 20 章：Docker、CI/CD 与部署

这一章解决什么：容器、健康检查、持续集成、环境配置与云端部署。

为什么岗位需要：作品必须能由他人按说明复现、测试并部署。

必须掌握

- Dockerfile
- Compose
- CI
- 健康检查

需要理解

- 滚动发布
- 迁移策略

章节实践

完成“可部署的 Agent Web 产品”。先运行 Day 01 最小示例，再按 本周计划 接入项目，最后使用 验收清单 检查。

常见风险

本地可运行但部署说明不完整。

面试自检

1. 容器化解决了什么问题？
2. 部署时如何管理数据库迁移？

本篇练习顺序

1. 先完成 W17 的 Day 01 最小示例。
2. 每周完成 WEEK_PLAN.md 和 CHECKLIST.md。
3. 每天把代码放在对应 exercises/day-XX/，把个人理解写在每日文件的保护区域。
4. 本篇结束时完成 W20 的项目里程碑，并写 REVIEW.md。

本篇完成标准

- [] 能不看教程解释本篇的核心概念。
- [] 能运行对应周的最小代码模板。
- [] 能说明一次失败、排查过程和改进。
- [] 项目 README、测试、验收和学习日志已更新。
- [] 能回答本篇周计划中的面试问题。

延伸阅读

- 24 周路线图
- 完整目录

- 本篇对应项目

第六篇：作品集与求职

本篇要解决的问题

把技术能力转译成招聘方能够快速验证的证据。

能力地图

周	主题	交付物	详细入口
W21	作品集工程化整理	三至四个可审阅的项目仓库	周计划
W22	Python、API 与数据库面试	基础面试答案库与两次模拟	周计划
W23	RAG、Agent 与系统设计面试	系统设计稿与限时开发演练	周计划
W24	简历、Demo 与投递	完整求职材料包和首轮投递	周计划

学习方法

本篇不要按“看完概念再集中编码”的方式学习。每个主题都要走完：概念理解 → 最小例子 → 接入项目 → 错误处理 → 测试或评测 → README 表达。

本篇核心观点

1. 从边界开始

先写清用户、输入、输出、权限和不做事项，再决定模型、框架或数据库。技术选择服务于业务流程，不是为了堆名词。

2. 从确定性部分开始

能用代码、SQL、Schema 或状态机明确表达的规则，不交给模型自由发挥。模型负责语言理解和候选生成，系统负责校验、权限、持久化和终止。

3. 从失败路径证明成熟度

每个阶段至少演示一个失败：空输入、错误类型、服务超时、检索无结果、工具拒绝、人工驳回或部署失败。真实工程能力体现在失败后系统仍然可控。

本篇项目连接

本篇主要推进：projects/project-04-capstone/。打开项目的 README.md、TASKS.md 和 ACCEPTANCE_CRITERIA.md，将章节知识转成可验收交付物。

本篇章节

第 21 章：作品集工程化整理

这一章解决什么：README、架构图、决策记录、截图、演示数据与仓库卫生。

为什么岗位需要：招聘方首先通过仓库结构判断工程成熟度。

必须掌握

- README
- 架构说明
- 运行指南
- 演示路径

需要理解

- ADR
- 版本发布

章节实践

完成“三至四个可审阅的项目仓库”。先运行 Day 01 最小示例，再按 本周计划 接入项目，最后使用 验收清单 检查。

常见风险

只展示最终界面，没有问题、约束和取舍。

面试自检

1. 优秀项目 README 应回答什么？
2. 如何解释技术债？

第 22 章：Python、API 与数据库面试

这一章解决什么：高频基础题、代码阅读、调试题、API 设计与 SQL。

为什么岗位需要：基础工程能力决定能否通过第一轮技术筛选。

必须掌握

- Python 数据模型
- HTTP
- SQL
- 测试

需要理解

- 性能权衡
- 并发基础

章节实践

完成“基础面试答案库与两次模拟”。先运行 Day 01 最小示例，再按 本周计划 接入项目，最后使用 验收清单 检查。

常见风险

背答案但无法结合项目举例。

面试自检

1. 如何排查慢 API？
2. 事务隔离级别如何选择？

第 23 章：RAG、Agent 与系统设计面试

这一章解决什么：架构题、容量估算、评测、安全、故障与模拟工作任务。

为什么岗位需要：中高级面试更关注边界、失败模式和可验证性。

必须掌握

- RAG 链路
- Agent 状态
- 评测
- 安全

需要理解

- 容量估算
- 故障演练

章节实践

完成“系统设计稿与限时开发演练”。先运行 Day 01 最小示例，再按 本周计划 接入项目，最后使用 验收清单 检查。

常见风险

只描述组件名，没有数据流和失败路径。

面试自检

1. 如何设计企业知识助手？
2. 如何控制 Agent 失控成本？

第 24 章：简历、Demo 与投递

这一章解决什么：技术简历、项目叙事、Demo 视频、GitHub 主页、行为面试与投递。

为什么岗位需要：把已完成能力翻译为招聘方能快速验证的证据。

必须掌握

- 成果量化
- 项目叙事
- Demo
- 投递检查

需要理解

- 岗位定制
- 跟进记录

章节实践

完成“完整求职材料包和首轮投递”。先运行 Day 01 最小示例，再按 本周计划 接入项目，最后使用 验收清单 检查。

常见风险

继续打磨而不开始投递。

面试自检

1. 请介绍一个最困难的技术取舍。
2. 为什么从设计转向 Agent 工程？

本篇练习顺序

1. 先完成 W21 的 Day 01 最小示例。
2. 每周完成 WEEK_PLAN.md 和 CHECKLIST.md。
3. 每天把代码放在对应 exercises/day-XX/，把个人理解写在每日文件的保护区。
4. 本篇结束时完成 W24 的项目里程碑，并写 REVIEW.md。

本篇完成标准

- [] 能不看教程解释本篇的核心概念。
- [] 能运行对应周的最小代码模板。
- [] 能说明一次失败、排查过程和改进。
- [] 项目 README、测试、验收和学习日志已更新。
- [] 能回答本篇周计划中的面试问题。

延伸阅读

- 24 周路线图
- 完整目录
- 本篇对应项目

项目实战篇

P1：设计需求分析助手

这个项目训练什么

把自然语言设计需求转换为可追踪的目标、约束、风险和待确认问题。

用户和业务流程

1. 提交原始需求
2. 校验并保存需求
3. 模型生成结构化分析
4. 调用项目规则工具
5. 用户确认或修订
6. 导出需求摘要

核心工程能力

- FastAPI 接口
- PostgreSQL 持久化
- JSON Schema
- Tool Calling
- 日志和统一异常
- 测试与 Docker

必须证明的结果

- 结构化字段有效率
- 人工修订率
- P95 API 延迟
- 失败请求率

风险边界

- 模型遗漏硬约束
- 敏感项目资料进入日志
- 重复提交产生重复记录

实施入口

- 项目 README
- 产品需求
- 架构说明
- 开发任务
- 验收标准
- 测试计划
- 安全说明
- 评测方案
- 部署说明

项目完成标准

不要只展示正常路径。至少演示一个输入错误、一个外部依赖失败、一个权限或安全边界，以及一个人工确认或明确的无答案结果。

P2：设计规范知识库 Agent

这个项目训练什么

让团队从多版本规范中获得带引用、可核查且受权限约束的答案。

用户和业务流程

1. 上传文档
2. 解析清洗与切块
3. 生成 Embedding 和索引
4. 查询改写与混合检索
5. Reranking
6. 生成带引用答案
7. 记录反馈与评测

核心工程能力

- 文档上传
- Chunking
- Embedding
- Metadata

- Hybrid Search
- Reranking
- 引用
- 评测与权限隔离

必须证明的结果

- Recall@5
- 引用正确率
- 无答案识别率
- 权限违规数

风险边界

- 过期规范被优先召回
- 跨团队数据泄漏
- 引用与答案不一致

实施入口

- 项目 README
- 产品需求
- 架构说明
- 开发任务
- 验收标准
- 测试计划
- 安全说明
- 评测方案
- 部署说明

项目完成标准

不要只展示正常路径。至少演示一个输入错误、一个外部依赖失败、一个权限或安全边界，以及一个人工确认或明确的无答案结果。

P3：智能设计评审 Agent

这个项目训练什么

依据目标、规范和证据完成多步骤评审，并在高风险结论前请求人工确认。

用户和业务流程

1. 接收评审任务
2. 解析目标 and 设计说明
3. 检索规范
4. 分别检查可用性、一致性和可访问性
5. 合并证据
6. 人工确认高风险项
7. 输出行动清单

核心工程能力

- 状态管理
- 条件分支
- 循环与终止
- 工具调用
- Human-in-the-loop
- 重试
- Tracing
- 成本统计

必须证明的结果

- 证据覆盖率
- 人工接受率
- 平均工具调用数
- 单次评审成本

风险边界

- 把主观偏好当作规则
- 工作流无限循环
- 评审证据不足

实施入口

- 项目 README
- 产品需求
- 架构说明
- 开发任务
- 验收标准
- 测试计划
- 安全说明
- 评测方案
- 部署说明

项目完成标准

不要只展示正常路径。至少演示一个输入错误、一个外部依赖失败、一个权限或安全边界，以及一个人工确认或明确的无答案结果。

P4：供应商风险审查 Agent

这个项目训练什么

自动整理供应商资料、检索政策、调用风险工具并生成需人工审批的审查建议。

用户和业务流程

1. 创建审查单
2. 解析供应商材料
3. RAG 检索内部政策
4. 调用制裁与财务风险工具
5. 生成风险分级
6. 人工审批
7. 写入审计记录并通知结果

核心工程能力

- 业务状态机
- RAG
- Tool Calling
- Agent 工作流

- 人工确认
- 最小权限
- 评测
- 容器化部署

必须证明的结果

- 高风险召回率
- 误报率
- 人工处理时长
- 审计记录完整率

风险边界

- 外部工具数据过期
- 模型结论被误当最终决定
- 敏感商业资料泄漏

实施入口

- 项目 README
- 产品需求
- 架构说明
- 开发任务
- 验收标准
- 测试计划
- 安全说明
- 评测方案
- 部署说明

项目完成标准

不要只展示正常路径。至少演示一个输入错误、一个外部依赖失败、一个权限或安全边界，以及一个人工确认或明确的无答案结果。

结语：从学习者到交付者

完成这套教程不代表掌握了所有 AI 技术，而代表你已经建立一套可以持续学习、构建、评测和交付的工作方法。接下来要做的是持续维护项目、跟踪官方文档、补充真实案例，并用面试和投递中的反馈反向改进作品集。